

2ÈME ANNÉE CYCLE D'INGÉNIEUR - SPÉCIALITÉ INFORMATIQUE.

MATIÈRE :
BASES DE DONNÉES AVANCÉES

Compte rendu TP3

Réalisé par :
BENMESSAOUD Abdellah

Encadrés par :
Mr. Samir Youcef

1 Introduction

Ce document présente un compte rendu détaillé basé sur une série de captures d'écran illustrant le développement et le test d'une application web utilisant le framework Spring Boot. L'application expose une API REST pour gérer des entités (initialement des étudiants, puis des adhérents) et interagit avec une base de données en mémoire H2.

2 Configuration Initiale et Premiers Endpoints

Cette section décrit la configuration initiale de l'application Spring Boot et la création des premiers endpoints REST simples.

2.1 Structure du Projet et Contrôleur Initial

L'image 1 montre la structure du projet dans l'IDE et la classe 'Monapi' annotée avec '@RestController'. Deux méthodes simples sont définies pour répondre aux requêtes GET sur les chemins '/b' et '/bn'.

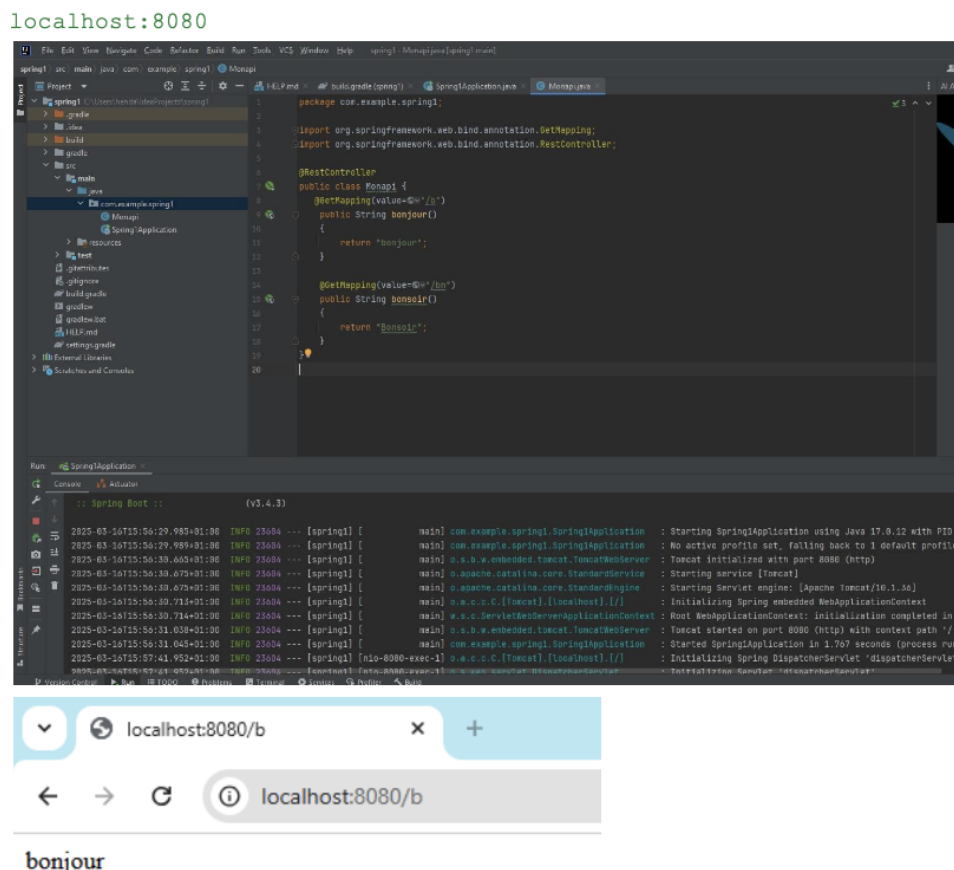


FIGURE 1 – Structure du projet et classe Monapi initiale.

Listing 1 – Extrait de Monapi.java (c1.png)

```
@RestController
```

```
public class Monapi {

    @GetMapping(value="/b")
    public String bonjour(){
        return "bonjour";
    }

    @GetMapping(value="/bn")
    public String bonsoir(){
        return "Bonsoir";
    }
    // ... (autres mthodes ajoutées plus tard)
}
```

2.2 Configuration du Port et Test

Le fichier ‘application.properties’ (Image 2) est utilisé pour configurer le port du serveur sur 9999. L’image montre également le résultat de l’appel à l’endpoint ‘/bn’ dans un navigateur, affichant "Bonsoir".

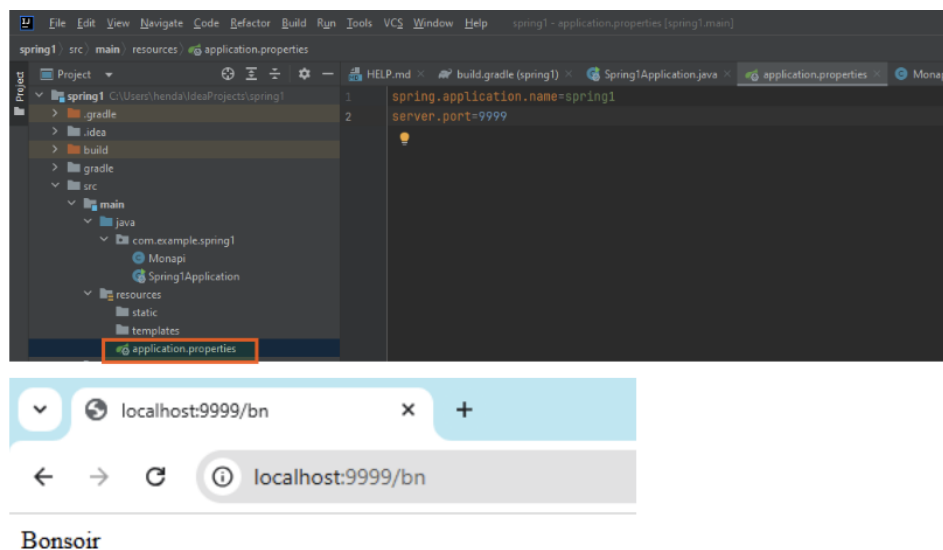


FIGURE 2 – Configuration du port et test de l’endpoint /bn.

Listing 2 – Extrait de application.properties (c2.png)

```
spring.application.name=spring1
server.port=9999
```

2.3 Gestion des Étudiants (En Mémoire)

L’application est ensuite étendue pour gérer une liste d’objets ‘Etudiant’ en mémoire.

2.3.1 Entité Etudiant

La classe 'Etudiant' est définie avec un 'id', un 'nom' et une 'moyenne' (Image 3).

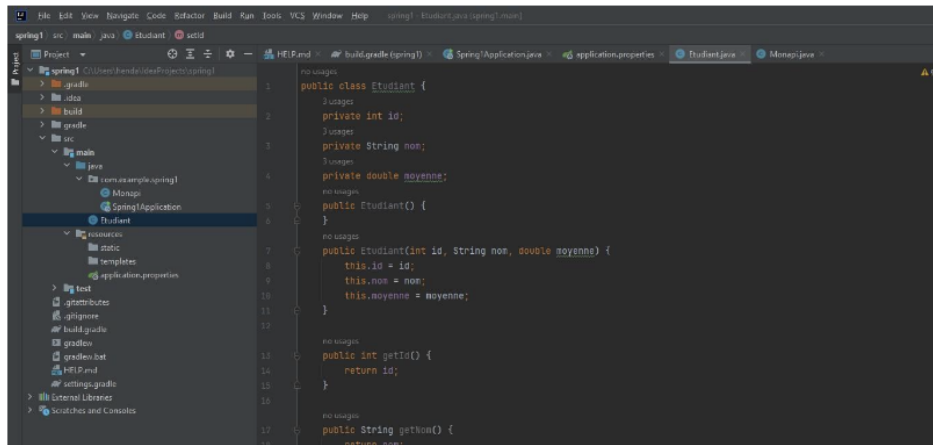


FIGURE 3 – Définition de la classe Etudiant.

Listing 3 – Extrait de Etudiant.java (c3.png)

```

public class Etudiant {
    private int id;
    private String nom;
    private double moyenne;

    // Constructeurs, Getters, Setters ...
}

```

2.3.2 Contrôleur Monapi (Suite)

La classe 'Monapi' est enrichie avec une liste statique d'étudiants et des méthodes pour les manipuler via des endpoints REST (Image 4). Un endpoint '/etudiant' renvoie un étudiant spécifique (ici, codé en dur pour l'exemple initial).

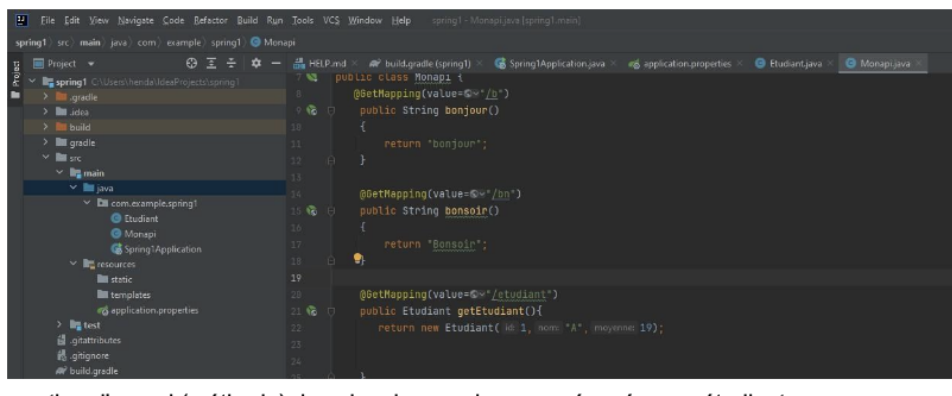


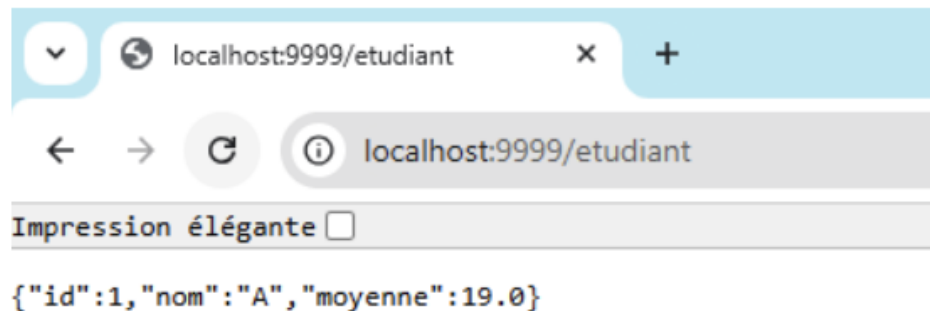
FIGURE 4 – Ajout de la gestion des étudiants dans Monapi.

Listing 4 – Extrait de Monapi.java (c4.png)

```
@RestController
public class Monapi {
    // ... autres mthodes

    @GetMapping(value="/etudiant")
    public Etudiant getEtudiant(){
        return new Etudiant(1, "nom": "A", moyenne: 19.0);
    }
    // ... (liste et autres mthodes ajoutées plus tard)
}
```

Le test de cet endpoint ‘/etudiant’ dans le navigateur est montré dans l’image 5, retournant l’objet étudiant au format JSON.



Je récupère un objet sous format JSON.
Le service web SOAP utilise le format XML.

FIGURE 5 – Test de l’endpoint /etudiant.

2.3.3 Opérations CRUD sur Étudiants

Des opérations CRUD complètes sont implémentées :

- **Lister tous les étudiants (‘/liste’)** : Une liste statique est initialisée et un endpoint ‘@GetMapping("/liste")’ est créé pour retourner cette liste (Image 6). Le test avec Postman et le navigateur montre le retour de la liste au format JSON (Image 7).
- **Obtenir un étudiant par ID (‘/getEtudiant’)** : L’endpoint ‘@GetMapping("/getEtudiant")’ prend un ‘id’ en paramètre et retourne l’étudiant correspondant de la liste (Image 8). Le test avec Postman et le navigateur pour ‘id=1’ est visible sur l’image 9.
- **Ajouter un étudiant (‘/addetudiant’)** : Un endpoint ‘@PostMapping("/addetudiant")’ permet d’ajouter un nouvel étudiant à la liste (Image 10). L’image 11 montre l’utilisation de Postman pour ajouter un étudiant et la vérification de la liste mise à jour.

- **Modifier un étudiant ('/modifier')** : Un endpoint '@PutMapping("/modifier")' permet de modifier le nom d'un étudiant existant via son 'id' (Image 12). Le test avec Postman est également montré.
- **Supprimer un étudiant ('/delete')** : L'image 11 montre également le code pour '@DeleteMapping("/delete")' et l'image 15 (présentée plus loin dans le contexte de Postman) montre un test de suppression.

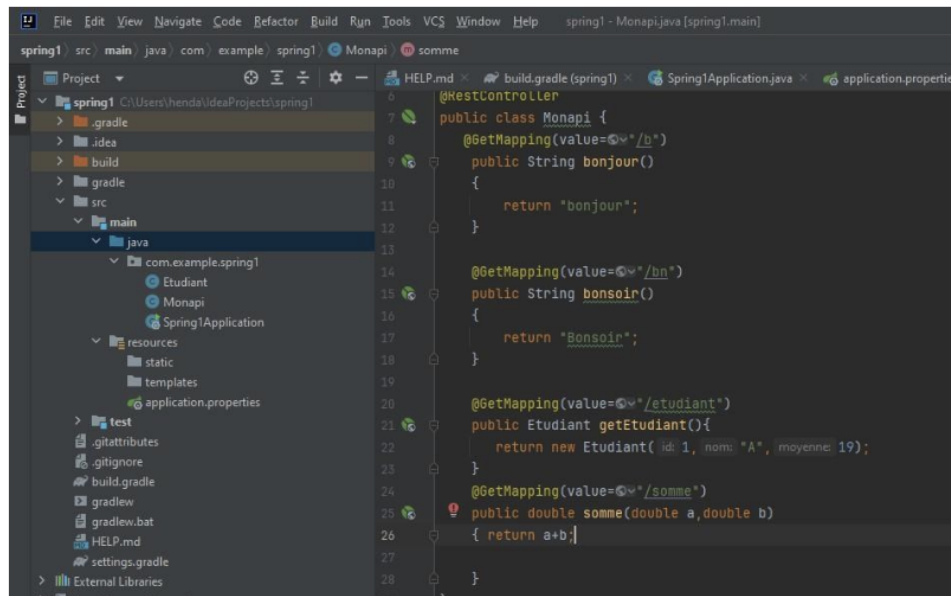


FIGURE 6 – Initialisation de la liste et endpoint /liste.

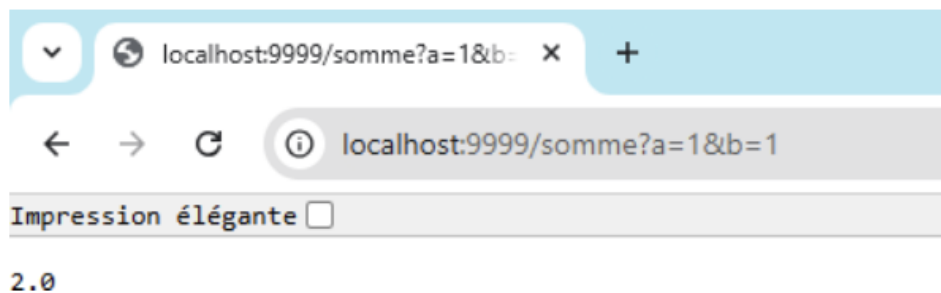


FIGURE 7 – Test de l'endpoint /liste avec navigateur et Postman.

2.4 Endpoint Additionnel : Somme

Un endpoint simple '/somme' est ajouté pour calculer la somme de deux nombres passés en paramètres (Image 13). Le test dans le navigateur est montré dans l'image 14.

2.5 Tests Complémentaires avec Postman

L'image 15 montre des tests supplémentaires avec Postman, notamment un GET sur '/liste' et un DELETE sur '/delete?id=1'.

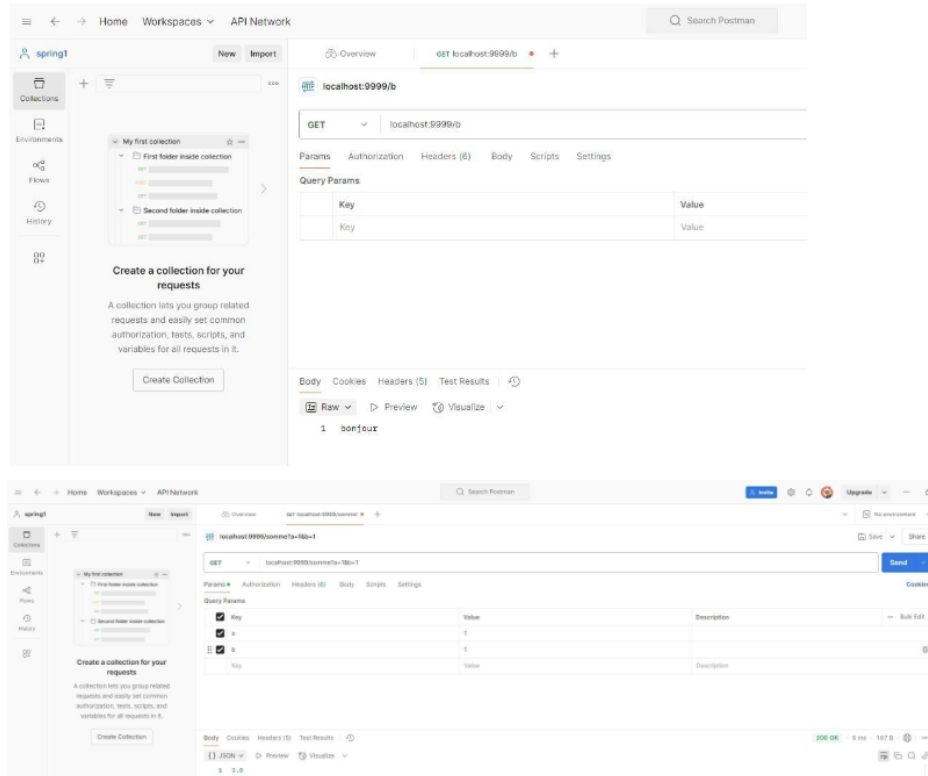


FIGURE 8 – Endpoint `/getEtudiant` par ID.

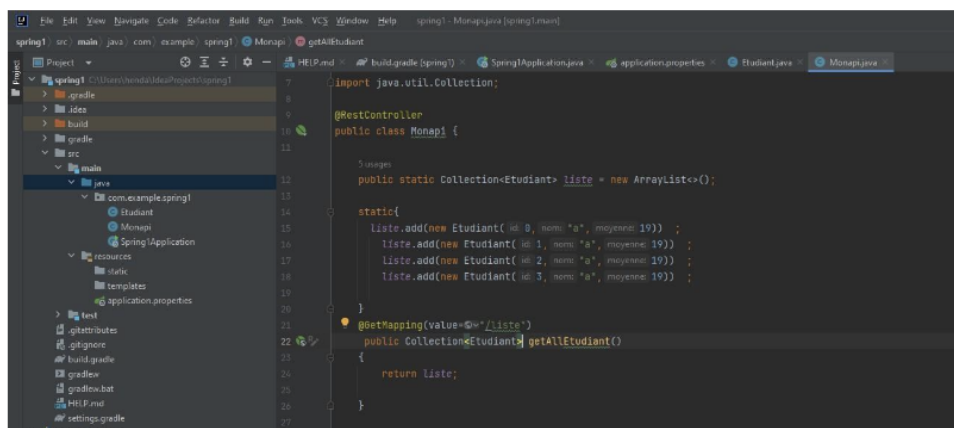


FIGURE 9 – Test de l'endpoint `/getEtudiant` avec navigateur et Postman.

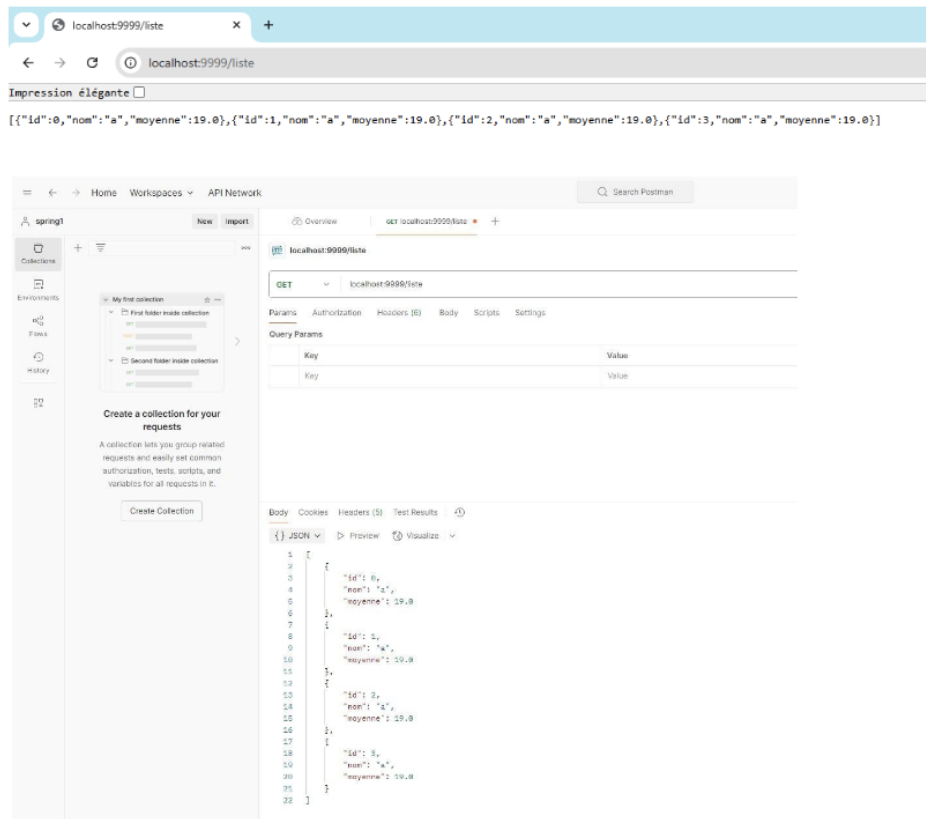
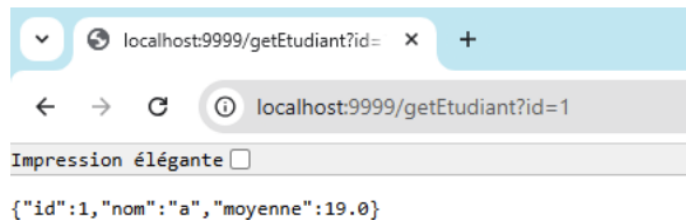


FIGURE 10 – Endpoint /addetudiant.



FIGURE 11 – Test POST /addetudiant, code DELETE /delete et test GET /liste après ajout.



test avec postman:

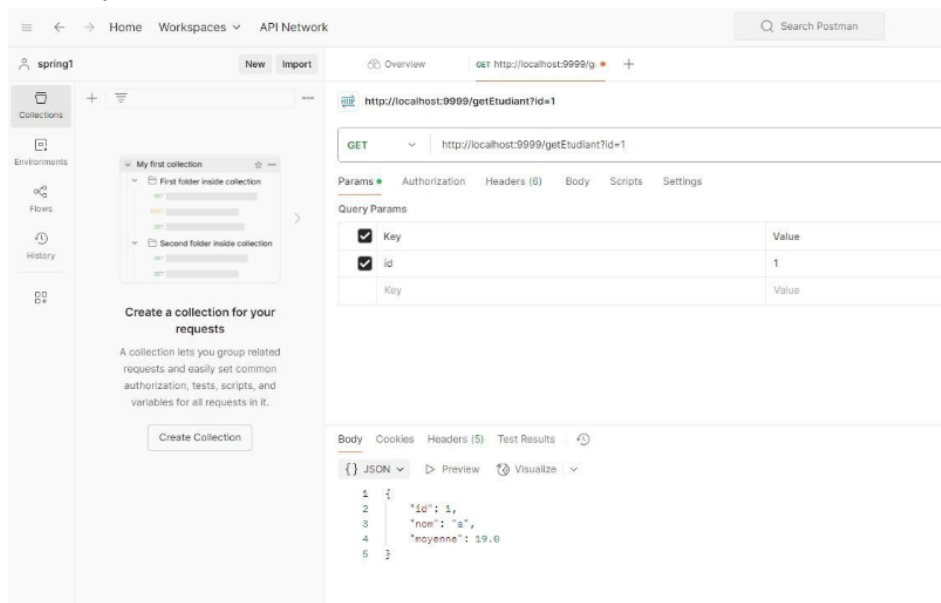


FIGURE 12 – Endpoint PUT /modifier et test avec Postman.

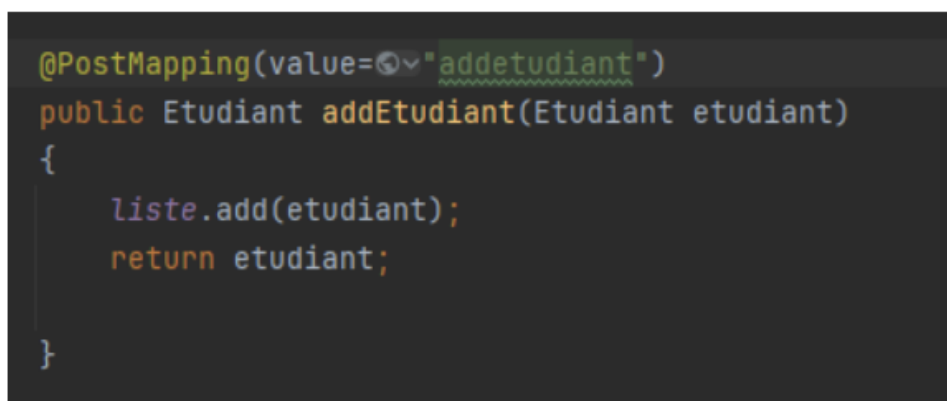


FIGURE 13 – Endpoint /somme.

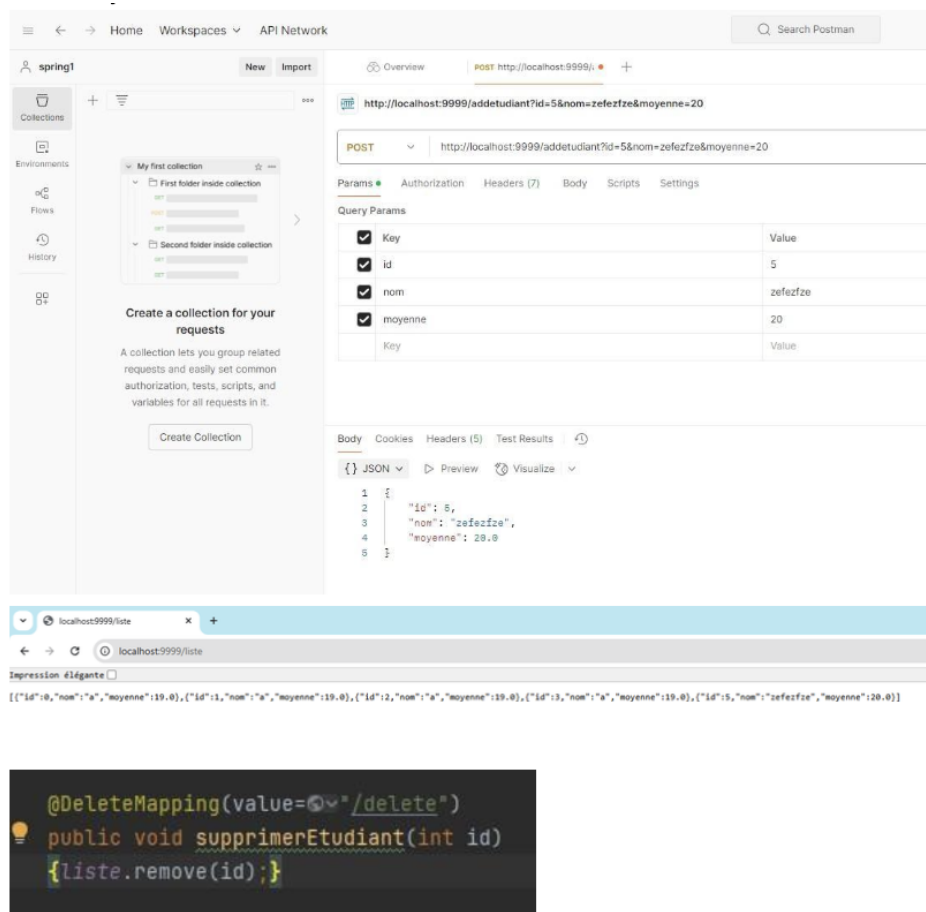


FIGURE 14 – Test de l'endpoint /somme.

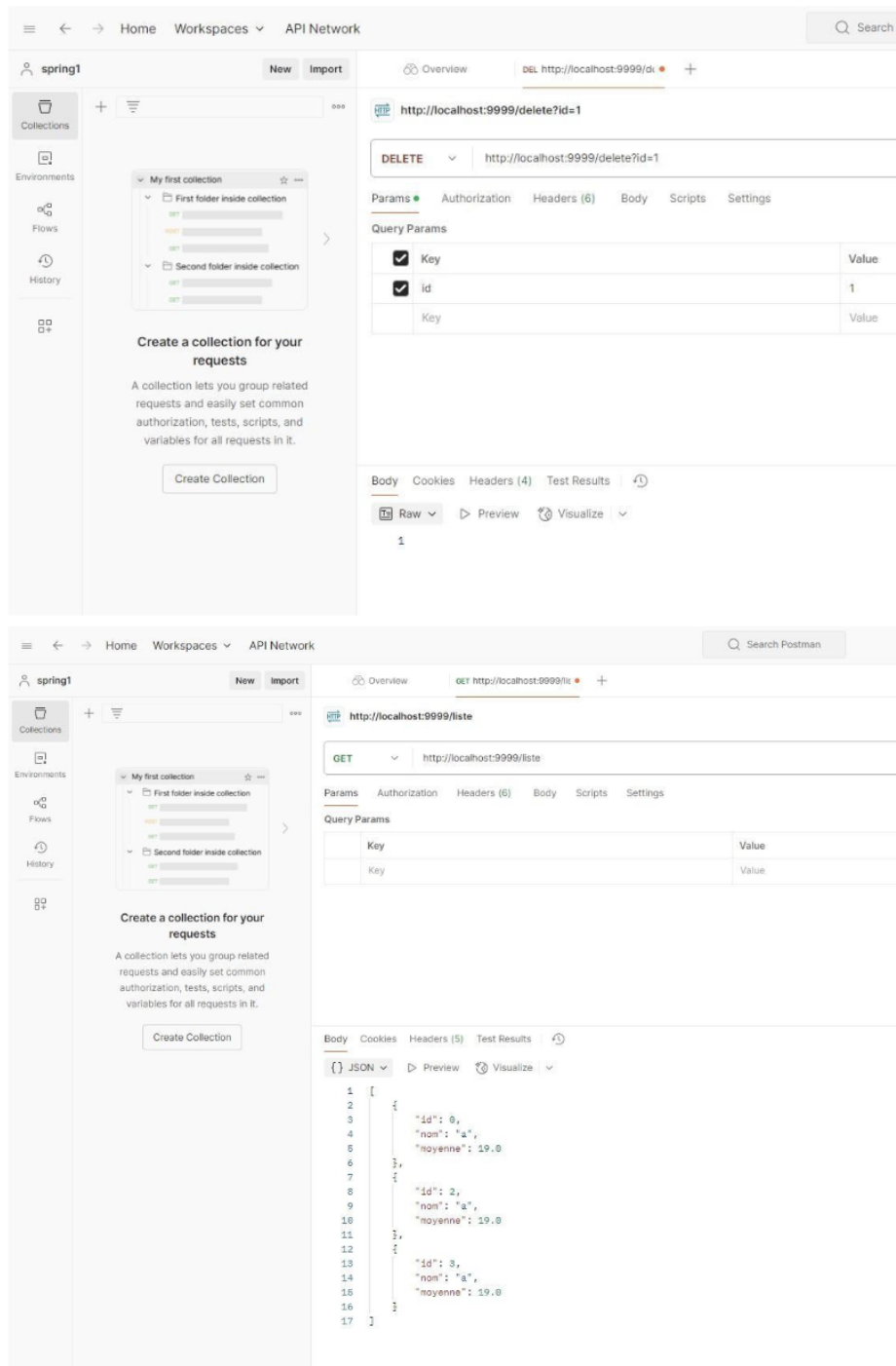


FIGURE 15 – Tests GET /liste et DELETE /delete avec Postman.

3 Intégration avec la Base de Données H2

L'application évolue ensuite pour utiliser une base de données en mémoire H2 afin de persister les données, en remplaçant la gestion en mémoire précédente. Une nouvelle entité 'Adherent' est introduite.

3.1 Entité Adherent et Repository

Une entité JPA 'Adherent' est définie, mappée à une table en base de données. Elle possède un 'id' (généré automatiquement), un 'nom', une 'ville' et un 'age' (Image 16). Bien que le Repository ne soit pas entièrement visible, sa présence est implicite pour l'interaction avec la base de données via Spring Data JPA.

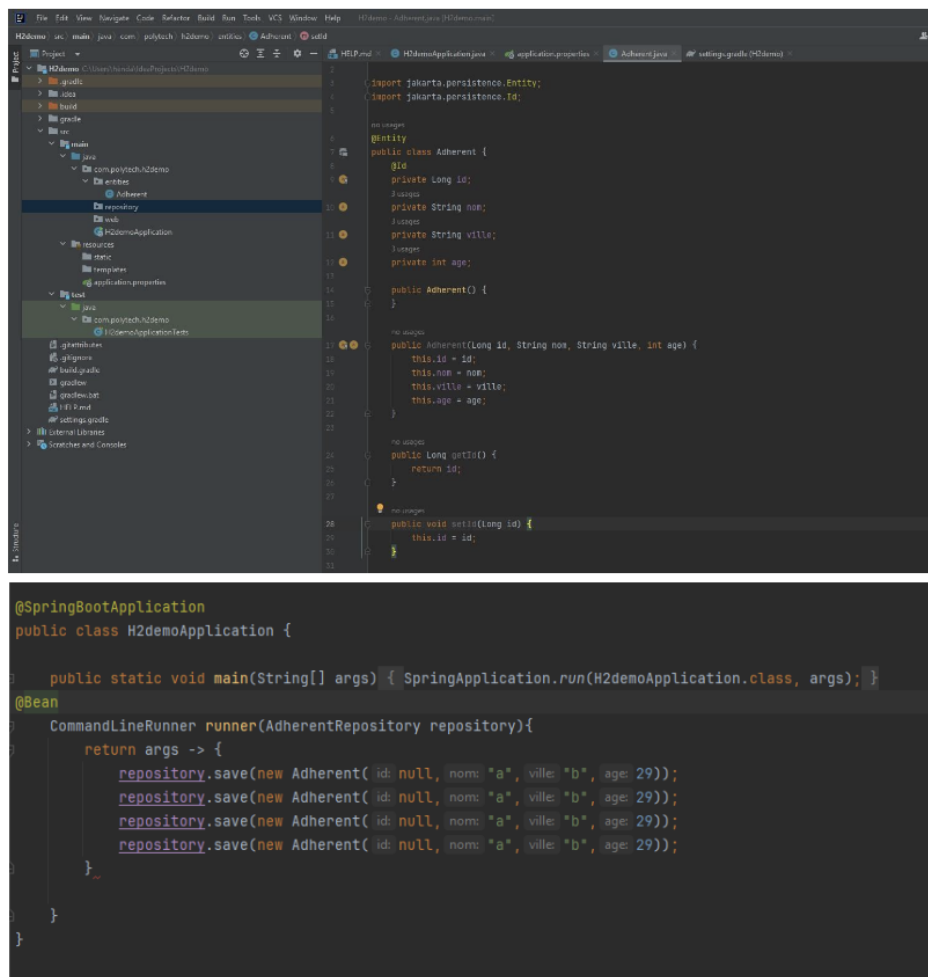


FIGURE 16 – Définition de l'entité Adherent.

Listing 5 – Extrait de Adherent.java (c17.png)

```

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

```

```
@Entity
public class Adherent {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    private String nom;
    private String ville;
    private int age;

    public Adherent() { }

    public Adherent(Long id, String nom, String ville, int age) {
        this.id = id;
        this.nom = nom;
        this.ville = ville;
        this.age = age;
    }
    // Getters et Setters ...
}
```

3.2 Configuration de la Base de Données H2

Le fichier ‘application.properties’ est mis à jour pour configurer la source de données H2 en mémoire (‘jdbc:h2:mem:adherent’) et activer la console H2 (‘spring.h2.console.enabled=true’) (Image 17).

Listing 6 – Extrait de application.properties (c18.png)

```
spring.application.name=H2demo
spring.datasource.url=jdbc:h2:mem:adherent
spring.h2.console.enabled=true
```

3.3 Initialisation des Données

L’image 16 montre également l’utilisation d’un ‘CommandLineRunner’ dans la classe principale de l’application (‘H2demoApplication’) pour insérer des données initiales dans la table ‘ADHERENT’ au démarrage de l’application, en utilisant le ‘AdherentRepository’ injecté.

Listing 7 – Extrait de H2demoApplication.java (c17.png)

```
@SpringBootApplication
public class H2demoApplication {

    public static void main(String[] args) {
        SpringApplication.run(H2demoApplication.class, args);
    }
}
```

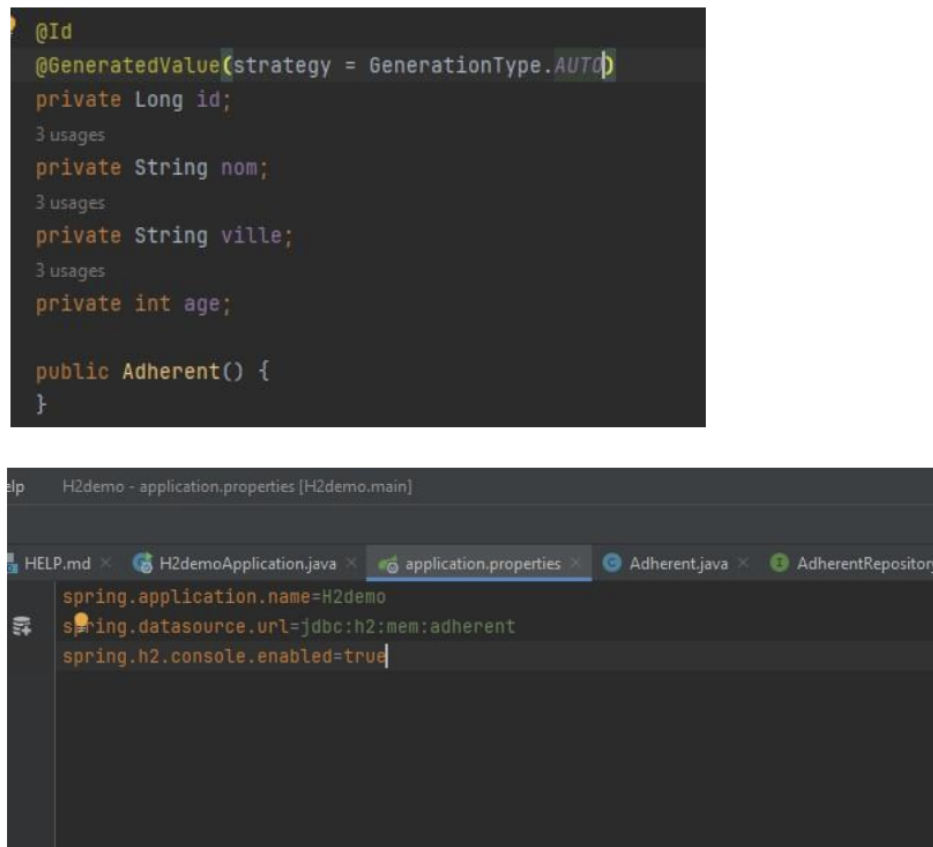


FIGURE 17 – Configuration H2 dans application.properties.

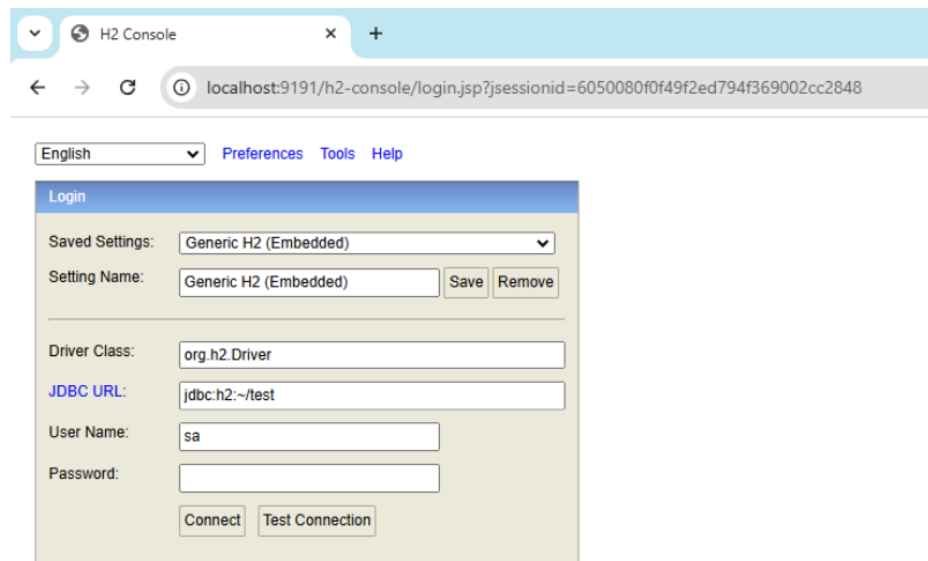
```
@Bean
CommandLineRunner runner(AdherentRepository repository){
    return args -> {
        repository.save(new Adherent(null, "a", "b", 29));
        repository.save(new Adherent(null, "a", "b", 29));
        repository.save(new Adherent(null, "a", "b", 29));
        repository.save(new Adherent(null, "a", "b", 29));
    };
}
```

3.4 Console H2

La console H2 permet d'inspecter le contenu de la base de données en mémoire via une interface web.

3.4.1 Connexion à la Console

L'image 18 montre l'écran de connexion de la console H2, accessible via le navigateur. Il faut utiliser le JDBC URL configuré ('jdbc:h2:mem:adherent') pour se connecter.



client qui permet de se connecter à une base h2

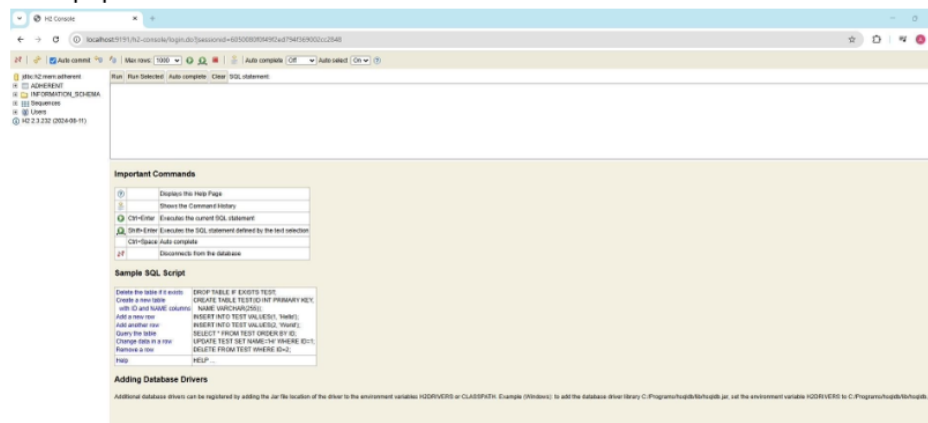


FIGURE 18 – Écran de connexion et interface de la console H2.

3.4.2 Interrogation de la Table Adherent

Une fois connecté, il est possible d'exécuter des requêtes SQL. L'image 19 montre l'exécution de 'SELECT * FROM ADHERENT;' dans la console H2, affichant les 4 enregistrements qui ont été insérés au démarrage de l'application par le 'CommandLine-Runner'.

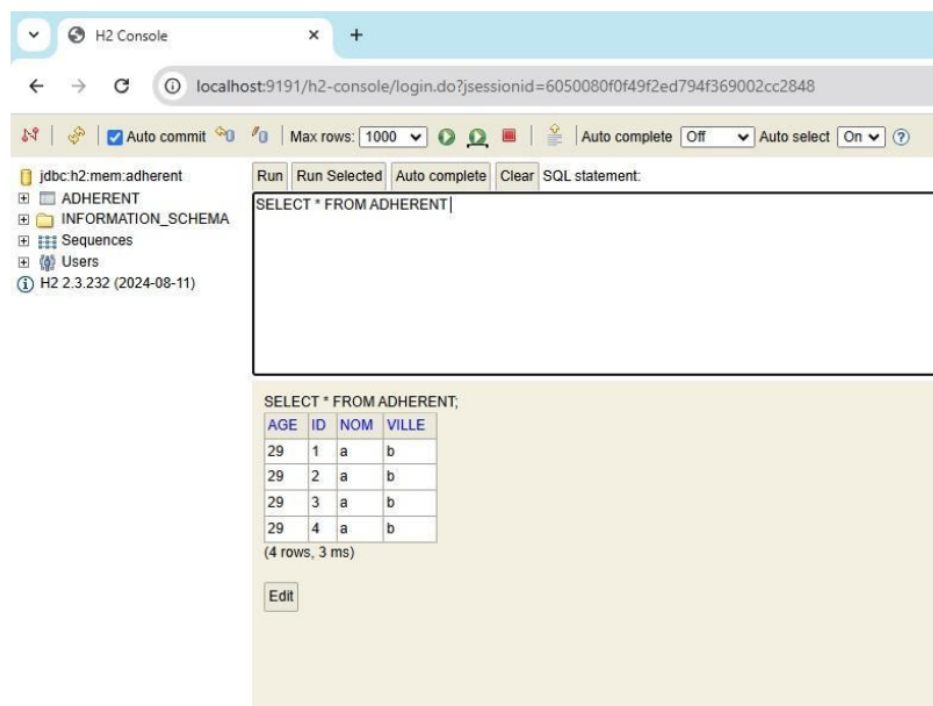


FIGURE 19 – Résultat de la requête `SELECT * FROM ADHERENT` dans la console H2.